

Part 8 - Configuring the Static Analysis Tool

1. `max-args` DESIGN

What it checks:

Checks if a function has too many parameters. The warning is `R0913 (too-many-arguments)`.

Default: 5

Decision: Change it to 7.

The default value of 5 was giving many warnings. For example:

- `create_assignment` → 10 parameters
- `update_profile` → 6 parameters
- `record_grade` → 8 parameters

Some functions in this project naturally need several values. For example, creating an assignment may require a title, subject, teacher, due date, maximum marks, etc.

Keeping the limit at 5 would create unnecessary warnings. Developers might then hide the warning or use `**kwargs`, which can make the code harder to understand.

2. `max-attributes` DESIGN

What it checks:

Checks if a class has too many instance variables. The warning is `R0902 (too-many-instance-attributes)`.

Default: 7

Decision: Change it to 10.

The tool found a test fixture in `test_export_service.py` with 11 attributes.

However, real model classes such as `User`, `Student`, and `Assignment` may naturally need many attributes to represent real objects.

Therefore, increasing the limit to 10 makes sense for normal domain models.

Importantly, the test fixture with **11 attributes will still produce a warning**. This means we do not completely hide the problem. The test fixture can later be divided into smaller, more focused fixtures.

3. **protected-access (W0212) CLASSES**

What it checks:

Checks when code accesses another class's protected/private-style attributes, such as:

```
grade._subject  
assignment._teacher_id
```

Default: Enabled.

Decision: Keep it enabled and do not add exclusions.

This warning appeared many times in the project.

It might be easy to remove the warnings by allowing names such as `_id`, `_email`, and `_created_at`. However, the manual review showed that these are **real design problems**, not false warnings.

Many services are directly accessing the internal data of model classes instead of using public methods or properties.

For example, instead of directly accessing:

```
grade._subject
```

the model should provide a proper public method or property.

4. **good-names BASIC**

What it checks:

Allows certain short variable names without giving an `invalid-name (C0103)` warning.

Default: `i`, `j`, `k`, `ex`, `Run`, `_`

Decision: Add `df`.

The tool reported `df` several times in `export_utils.py`.

In Python and pandas, `df` is a very common name for a **DataFrame**. Most Python developers immediately understand what `df` means.

Therefore, I added `df` to the allowed names.

5. `min-similarity-lines` SIMILARITIES

What it checks:

Checks how many consecutive lines must be similar before Pylint reports duplicate code.

Default: 4

Decision: Keep the default.

The manual review found some duplicated code in `export_service.py` and `grade_service.py`.

However, Pylint did not detect this duplication because the code used different variable names even though the overall logic was very similar.

Lowering the threshold could create many unnecessary warnings for normal repeated code.

This is a limitation of the tool. More complex duplication should be found through **manual code review** rather than changing the Pylint setting.

6. `max-statements` DESIGN

What it checks:

Checks if a function contains too many statements. The warning is `R0915 (too-many-statements)`.

Default: 50

Decision: Keep the default.

`AuthService.register_user` had **53 statements**, so Pylint correctly flagged it.

The manual review also found that this function was doing too many different jobs, including:

- Validation
- Creating users

- Saving data
- Sending notifications
- Logging

Therefore, the warning represents a real problem.

I will **keep the limit at 50** instead of increasing it. The better solution is to divide `register_user` into smaller functions, such as:

```
_validate_registration()  
_create_user_by_type()  
_notify_new_user()
```

This will make the code easier to read, test, and maintain.